SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand

6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand

9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

---

Code of Conduct:

I R . I n d h u (1 9 2 5 2 4 3 3 2) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Indhu

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues

Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q1. Debug CREATE TABLE Statement

Example Incorrect SQL

CREATE TABLE Student

(

StudentID INT,

StudentName VARCHAR(50)

Age INT,

PRIMARY KEY StudentID

);

Errors

- Missing comma after StudentName.
- PRIMARY KEY syntax is incorrect.
- StudentID should be NOT NULL.

```
mysql> CREATE TABLE Student
    -> (
    -> StudentID INT,
    -> StudentName VARCHAR(50)
    -> Age INT,
    -> PRIMARY KEY StudentID
    -> );
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server
version for the right syntax to use near 'Age INT,
PRIMARY KEY StudentID
)' at line 5
```

Correct SQL

CREATE TABLE Student

(

StudentID INT NOT NULL,

StudentName VARCHAR(50),

Age INT,

PRIMARY KEY (StudentID));

```
mysql> CREATE TABLE Student
    -> (
    -> StudentID INT NOT NULL,
    -> StudentName VARCHAR(50),
    -> Age INT,
    -> PRIMARY KEY (StudentID)
    -> );
Query OK, 0 rows affected (0.11 sec)
```

Explanation

Added the missing comma, corrected the PRIMARY KEY syntax, and made the primary key non-null.

Q2. Aggregate Function Without GROUP BY

Incorrect SQL

SELECT DeptID, AVG(Salary)

FROM Employee;

Error

Aggregate function is used with a normal column but no GROUP BY.

```
mysql> SELECT DeptID, AVG(Salary)
    -> FROM Employee;
ERROR 1140 (42000): In aggregated query without GROUP BY, expression #1 of SELECT list contains nonaggregated col
umn 'gojo.Employee.DeptID'; this is incompatible with sql_mode=only_full_group_by
```

Correct SQL

SELECT DeptID, AVG(Salary)

FROM Employee

GROUP BY DeptID;

```
mysql> SELECT DeptID, AVG(Salary)
    -> FROM Employee
    -> GROUP BY DeptID;
+--------+--------------+
| DeptID | AVG(Salary)  |
+--------+--------------+
|      1 | 52500.000000 |
|      2 | 55000.000000 |
|      3 | 50000.000000 |
+--------+--------------+
3 rows in set (0.05 sec)
```

Explanation

GROUP BY groups employees according to department before calculating the average salary.

Q3. Incorrect SELECT Statement

Incorrect SQL

SELECT StudntName, Ag

FROM Student;

Errors

- StudntName is misspelled.

- Ag is misspelled.

```
mysql> SELECT StudntName, Ag
    -> FROM Student;
ERROR 1054 (42S22): Unknown column 'StudntName' in 'field list'
```

Correct SQL

SELECT StudentName, Age

FROM Student;

```
mysql> SELECT StudentName, Age
    -> FROM Student;
+-------------+------+
| StudentName | Age  |
+-------------+------+
| Rahul       |   20 |
| Priya       |   21 |
| Kiran       |   22 |
| Sneha       |   20 |
+-------------+------+
4 rows in set (0.00 sec)
```

Explanation

Corrected the column names.

Q4. Incorrect JOIN

Incorrect SQL

SELECT Student.StudentName, Department.DeptName

FROM Student

JOIN Department;

Error

Missing JOIN condition.

```
mysql> SELECT Student.StudentName,
    -> Department.DeptName
    -> FROM Student
    -> JOIN Department
    -> ON Student.DeptID = Department.DeptID;
+-------------+----------+
| StudentName | DeptName |
+-------------+----------+
| Rahul       | CSE      |
| Priya       | IT       |
| Kiran       | CSE      |
| Sneha       | ECE      |
+-------------+----------+
4 rows in set (0.04 sec)
```

Correct SQL

SELECT Student.StudentName,

Department.DeptName

FROM Student

JOIN Department

ON Student.DeptID = Department.DeptID;

Explanation

Added the proper join condition.

Q5. Subquery Returns Multiple Values

Incorrect SQL

SELECT StudentName

FROM Student

WHERE DeptID =

(

SELECT DeptID

FROM Department);

Error

Subquery returns multiple rows.

```
mysql> SELECT StudentName
    -> FROM Student
    -> WHERE DeptID =
    -> (
    -> SELECT DeptID
    -> FROM Department
    -> );
ERROR 1242 (21000): Subquery returns more than 1 row
```

Correct SQL

SELECT StudentName

FROM Student

WHERE DeptID IN

(

SELECT DeptID

FROM Department);

```
mysql> SELECT StudentName
    -> FROM Student
    -> WHERE DeptID IN
    -> (
    -> SELECT DeptID
    -> FROM Department
    -> );
+-------------+
| StudentName |
+-------------+
| Rahul       |
| Priya       |
| Kiran       |
| Sneha       |
+-------------+
4 rows in set (0.00 sec)
```

Explanation

IN is used because multiple department IDs are returned.

Q6. Debug VIEW Creation

Incorrect SQL

CREATE VIEW StudentView AS

SELECT StudentID, Name

FROM Student;

Error

Column Name does not exist.

```
mysql> CREATE VIEW StudentView AS
    -> SELECT StudentID, Name
    -> FROM Student;
ERROR 1054 (42S22): Unknown column 'Name' in 'field list'
```

Correct SQL

CREATE VIEW StudentView AS

SELECT StudentID, StudentName

FROM Student;

```
mysql> CREATE VIEW StudentView AS
    -> SELECT StudentID, StudentName
    -> FROM Student;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM StudentView;
+-----------+-------------+
| StudentID | StudentName |
+-----------+-------------+
|       101 | Rahul       |
|       102 | Priya       |
|       103 | Kiran       |
|       104 | Sneha       |
+-----------+-------------+
4 rows in set (0.00 sec)
```

Explanation

Used the correct column name.

Q7. Optimize WHERE Clause

Incorrect SQL

SELECT *

FROM Employee

WHERE Salary > 5000

AND Salary > 3000;

```
mysql> SELECT *
    -> FROM Employee
    -> WHERE Salary > 5000
    -> AND Salary > 3000;
+-------+---------+----------+--------+
| EmpID | EmpName | Salary   | DeptID |
+-------+---------+----------+--------+
|     1 | Ravi    | 45000.00 |      1 |
|     2 | Priya   | 55000.00 |      2 |
|     3 | Kiran   | 60000.00 |      1 |
|     4 | Anitha  | 50000.00 |      3 |
+-------+---------+----------+--------+
4 rows in set (0.01 sec)
```

Error

Second condition is unnecessary.

Correct SQL

SELECT *

FROM Employee

WHERE Salary > 5000;

```
mysql> SELECT *
    -> FROM Employee
    -> WHERE Salary > 5000;
+-------+---------+----------+--------+
| EmpID | EmpName | Salary   | DeptID |
+-------+---------+----------+--------+
|     1 | Ravi    | 45000.00 |      1 |
|     2 | Priya   | 55000.00 |      2 |
|     3 | Kiran   | 60000.00 |      1 |
|     4 | Anitha  | 50000.00 |      3 |
+-------+---------+----------+--------+
4 rows in set (0.00 sec)
```

Explanation

If salary is greater than 5000, it is automatically greater than 3000.

Q8. Integrity Constraint Violation

Incorrect SQL

INSERT INTO Student

VALUES

(NULL,'Rahul',20);

Error

Primary key cannot be NULL.

```
mysql> INSERT INTO Student
    -> VALUES
    -> (NULL,'Rahul',20,1);
ERROR 1048 (23000): Column 'StudentID' cannot be null
```

Correct SQL

INSERT INTO Student

VALUES

(101,'Rahul',20);

```
mysql> INSERT INTO Student
    -> VALUES
    -> (105,'Arun',21,2);
Query OK, 1 row affected (0.01 sec)
```

Explanation

Provided a valid primary key.

Q9. Fix 1NF / 2NF Violation

Incorrect Table

StudentID	StudentName	Subjects
101	Ravi	DBMS, Java

Error

Multiple values are stored in one column (violates 1NF).

Correct Design

Student

StudentID	StudentName
101	Ravi

StudentSubject

StudentID	Subject
101	DBMS
101	Java

```
mysql> CREATE TABLE StudentSubjects
    -> (
    -> StudentID INT,
    -> StudentName VARCHAR(50),
    -> Subjects VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO StudentSubjects
    -> VALUES
    -> (101,'Rahul','DBMS,Java');
Query OK, 1 row affected (0.01 sec)

mysql> CREATE TABLE Student_Subject
    -> (
    -> StudentID INT,
    -> SubjectName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Student_Subject
    -> VALUES
    -> (101,'DBMS'),
    -> (101,'Java');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

Explanation

Each cell now contains only one value.

Q10. Normalize to 3NF

Incorrect Relation

StudentID	StudentName	DeptID	DeptName

Error

DeptName depends on DeptID, causing redundancy.

Correct Design

Student

StudentID StudentName DeptID

Department

DeptID	DeptName

```
mysql> CREATE TABLE Student_Normalized
    -> (
    -> StudentID INT PRIMARY KEY,
    -> StudentName VARCHAR(50),
    -> DeptID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Department_Normalized
    -> (
    -> DeptID INT PRIMARY KEY,
    -> DeptName VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Explanation

Department information is stored separately, eliminating redundancy and satisfying 3NF.